

CODE_REVIEW_AND_GAP_ANALYSIS

Helix OTA — Code Review & Gap Analysis

Document ID: HELOTA-REVIEW-001 **Version:** 1.0.0 **Status:** Active **Date:** 2026-03-05 **Scope:** Full documentation suite for 1.0.0-MVP **Reviewer:** Independent Code Review

Executive Summary

A comprehensive review of all Helix OTA documentation reveals **11 CRITICAL gaps** that would block development, **9 HIGH-priority issues** that should be resolved early, **8 MEDIUM-priority issues**, and **7 LOW-priority items**. The most severe finding is **pervasive inconsistency** between documents: API endpoints, data models, enum values, and field names differ significantly across the VERSION_ROADMAP, REST_API_SPECIFICATION, DATABASE_SCHEMA, SYSTEM_ARCHITECTURE, and SERVER_IMPLEMENTATION. A development team cannot begin implementation without resolving these contradictions.

Verdict: Development CANNOT start until all CRITICAL issues are resolved.

1. CRITICAL Issues — Must Fix Before Development Begins

C1. Device Registration API — Endpoint Path Contradiction

Document	Endpoint
VERSION_ROADMAP §4.3	POST /api/v1/devices
REST_API_SPECIFICATION §3.1	POST /api/v1/devices/register
NEW_SUBMODULE_SPECIFICATIONS §2.4	POST /api/v1/devices/register

Impact: A client built against one document will 404 against a server built against the other. This is a showstopper.

Recommendation: Adopt `POST /api/v1/devices/register` (with mTLS auth context) as the canonical path. The bare `POST /api/v1/devices` should be reserved for the admin list/create endpoint. Update `VERSION_ROADMAP` to match.

C2. Device Registration Request Body — Fundamentally Different Schemas

Document	Fields
VERSION_ROADMAP §4.7	serial, model, current_version, slot_suffix, hardware_fingerprint
REST_API_SPECIFICATION §3.1	device_id, hardware_model, firmware_version, os_type, serial_number, metadata
SYSTEM_ARCHITECTURE §3.1.2	Serial, Model, CurrentVersion, SlotSuffix, HardwareFingerprint

Impact: The `REST_API_SPECIFICATION` includes `device_id` (client-generated) and `os_type` but omits `slot_suffix` and `hardware_fingerprint`. The `VERSION_ROADMAP` omits `device_id` and `os_type`. The `SYSTEM_ARCHITECTURE` matches the `ROADMAP` but conflicts with the API spec. A device registering with one schema will have missing required fields on the server.

Recommendation: Define ONE canonical `DeviceRegistrationRequest` with all required fields: - `device_id` (client-generated, validated against mTLS CN) - `hardware_model` (renamed from `model`) - `serial_number` (renamed from `serial`) - `firmware_version` (renamed from `current_version`) - `os_type` (required for server routing) - `slot_suffix` (required for A/B update logic) - `hardware_fingerprint` (required for duplicate detection)

Update ALL documents to use this single schema.

C3. Update Check API — Method AND Path Contradiction

Document	Method	Path
VERSION_ROADMAP §4.3, §4.7	GET	<code>/api/v1/devices/{device_id}/update-check</code>
REST_API_SPECIFICATION §5.1	POST	<code>/api/v1/updates/check</code>
SYSTEM_ARCHITECTURE §3.1.1	GET	

Document	Method	Path
SERVER_IMPLEMENTATION §3.1	N/A	/api/v1/devices/{deviceID}/update-check CheckForUpdate(ctx, req)

Impact: GET vs POST is a fundamental semantic difference. The path structure (device-scoped vs flat) determines routing and auth middleware. Two documents say GET with device-scoped path; the REST spec says POST with a flat path and request body.

Recommendation: Use POST /api/v1/updates/check as the canonical endpoint (the device sends context in the body; POST avoids URL-encoding sensitive parameters and allows richer request bodies for future delta queries). Update VERSION_ROADMAP and SYSTEM_ARCHITECTURE to match.

C4. Artifact Upload API — Path Contradiction

Document	Endpoint
VERSION_ROADMAP §4.3	POST /api/v1/artifacts
REST_API_SPECIFICATION §4.1	POST /api/v1/artifacts/upload
NEW_SUBMODULE_SPECIFICATIONS §2.4	POST /api/v1/artifacts/upload

Impact: Same as C1 — 404 mismatch.

Recommendation: Adopt POST /api/v1/artifacts/upload (explicit action path). Update VERSION_ROADMAP.

C5. Rollout Status Enum — Three Different Enum Sets

Document	Values
VERSION_ROADMAP §4.3	ACTIVE, PAUSED, COMPLETED, ROLLED_BACK
DATABASE_SCHEMA rollout_status_enum	draft, paused, active, completed, halted, rolled_back
SERVER_IMPLEMENTATION RolloutStatus	DRAFT, RUNNING, PAUSED, HALTED, COMPLETED

Impact: The VERSION_ROADMAP is missing DRAFT and HALTED. The SERVER_IMPLEMENTATION uses RUNNING instead of ACTIVE. The database has all six values. The state machine diagram in SERVER_IMPLEMENTATION §4.1 shows DRAFT→RUNNING transitions that are not in the VERSION_ROADMAP at all.

Recommendation: Adopt the DATABASE_SCHEMA enum as canonical (it's the most complete): draft, active, paused, halted, completed, rolled_back. Define a mapping to wire-format strings (e.g., DRAFT, ACTIVE, etc.) in the REST API spec. Update all other documents.

C6. Device Status Enum — Three Different Enum Sets

Document	Values
REST_API_SPECIFICATION §3.1-3.5	registered, online, offline, decommissioned
DATABASE_SCHEMA device_status_enum	online, offline, updating, error
VERSION_ROADMAP §4.4	DOWNLOADING, DOWNLOAD_PAUSED, VERIFYING, etc. (these are update lifecycle states, not device states)

Impact: The API spec includes registered and decommissioned which don't exist in the database enum. The database includes updating and error which don't exist in the API spec. decommissioned is handled via soft-delete (deleted_at IS NOT NULL) in the database, not a status value.

Recommendation: Reconcile as follows: - Database enum: online, offline, updating, error - decommissioned = soft-deleted row (deleted_at IS NOT NULL), NOT a status value - registered = initial state before first check-in (= offline) - Update REST_API_SPECIFICATION to use the database enum values

C7. OS Type Enum — CHECK Constraint vs API Spec Mismatch

Document	Allowed Values
REST_API_SPECIFICATION §3.1	linux-arm64, linux-amd64, rtos-armv7
DATABASE_SCHEMA chk_devices_os_type	android, linux, windows

Impact: An Android device registering with os_type: "android" would pass the DB constraint but fail the API validation. Conversely, linux-arm64 would pass API validation but violate the DB CHECK constraint. This would cause 500 errors on registration.

Recommendation: The database CHECK constraint should be removed or expanded to include the full set. Define a canonical enum: android, linux-arm64, linux-amd64, windows, rtos-armv7. The API validates against this enum; the DB stores it as a VARCHAR with a relaxed CHECK (or no CHECK, relying on application-level validation).

C8. Validation Status Values — Three Different Sets

Document	Values
SYSTEM_ARCHITECTURE §3.1.4	PENDING, INVALID, VALID
DATABASE_SCHEMA upload_status_enum	uploading, validating, ready, failed
REST_API_SPECIFICATION §4.1	passed, failed, pending

Impact: Client expecting VALID receives ready or passed. Filter queries using wrong value return empty results.

Recommendation: Adopt DATABASE_SCHEMA values as canonical (uploading → validating → ready | failed) and define wire-format mapping in REST API: uploading → "pending", validating → "validating", ready → "passed", failed → "failed".

C9. Artifact Validation Chain — Order Contradiction

Document	Order
SYSTEM_ARCHITECTURE §3.1.4, SECURITY_ARCHITECTURE §4.4	Hash → Signature → Structure → Compatibility
SERVER_IMPLEMENTATION §5.1	Structure → Hash → Signature → Compatibility

Impact: Validation chain order is safety-critical. Hash must come first because: (1) it's the fastest check, (2) if the hash is wrong, the signature is meaningless (it signs the hash), (3) checking structure of a corrupted file wastes resources. Starting with structure check means a corrupted file could partially pass validation before hash catches it.

Recommendation: Mandate Hash → Signature → Structure → Compatibility as the ONLY valid order. This is the correct dependency chain: hash validates integrity, signature validates authenticity, structure validates format, compatibility validates target. Update SERVER_IMPLEMENTATION to match.

C10. Go Module Path — Two Different Module Identifiers

Document	Module Path
SERVER_IMPLEMENTATION §2	github.com/vasic-digital/ helix-ota-server
NEW_SUBMODULE_SPECIFICATIONS §2.3	dev.helix.ota.server

Impact: Go modules cannot be imported if the path doesn't match. A developer following SERVER_IMPLEMENTATION would create go.mod with one path; a developer following NEW_SUBMODULE_SPECIFICATIONS would use the other. The code won't compile if submodules reference the wrong path.

Recommendation: Choose one path convention and update all documents. Recommend `dev.helix.ota.server` (follows the `NEW_SUBMODULE_SPECIFICATIONS` convention which is more deliberate and matches the organizational namespace). Update `SERVER_IMPLEMENTATION` `go.mod`.

C11. Primary Key Convention — UUID vs Prefixed String IDs

Document	ID Format
DATABASE_SCHEMA §1.2	UUID v4 (<code>gen_random_uuid()</code>)
VERSION_ROADMAP §4.7	Prefixed strings: <code>dev_01HXYZ...</code> , <code>art_01HABC...</code> , <code>rol_...</code>
REST_API_SPECIFICATION	Prefixed strings: <code>dev_01HWIDGET001</code> , <code>art_01HART003</code>
SYSTEM_ARCHITECTURE	<code>generateID("dev_")</code> producing prefixed IDs

Impact: The database schema uses raw UUIDs as primary keys. The API and code use prefixed string IDs (like ULID or custom format). The mapping between these two is never defined. The devices table has both `id` UUID and `device_id` VARCHAR(256) — which is the “real” identifier used in API paths?

Recommendation: Define clearly: - `id` UUID = internal database primary key, never exposed in API - `device_id` VARCHAR(256) = external-facing identifier, used in all API paths, uniquely indexed - All API paths and responses use the `device_id` (prefixed string), not the UUID - Apply the same pattern to artifacts (`artifact_id`), rollouts (`rollout_id`), etc. - Document the ID generation format (e.g., ULID with prefix: `dev_01HXYZ...`)

2. HIGH Issues — Should Fix in Early Development

H1. No User Management API Endpoints Defined

The RBAC matrix in `SECURITY_ARCHITECTURE` §5.4 lists user CRUD operations (create user, update role, delete user, view user list), but the `REST_API_SPECIFICATION` contains NO endpoints for user management. Only auth endpoints (login, refresh, logout, TOTP) are defined.

Recommendation: Add the following endpoints to `REST_API_SPECIFICATION`: - `POST /api/v1/users` — Create user (admin only) - `GET /api/v1/users` — List users (admin only) - `GET /api/v1/users/{id}` — Get user (admin only) - `PATCH /api/v1/users/{id}` — Update user role/status (admin only) - `DELETE /api/v1/users/{id}` — Delete user (admin only)

H2. No Device Group Management API Endpoints Defined

The `device_groups` table exists in `DATABASE_SCHEMA`, and `PUT /api/v1/devices/{device_id}/group` exists in `REST_API_SPECIFICATION` for device-to-group assignment, but there are NO endpoints for creating, reading, updating, or deleting device groups themselves.

Recommendation: Add Device Group CRUD endpoints: - `POST /api/v1/device-groups` — Create group - `GET /api/v1/device-groups` — List groups - `GET /api/v1/device-groups/{id}` — Get group details - `PATCH /api/v1/device-groups/{id}` — Update group (name, filter rules) - `DELETE /api/v1/device-groups/{id}` — Delete group

H3. No API Key Management Endpoints Defined

`SECURITY_ARCHITECTURE` §5.5 defines API key format, hashing, scoping, and expiration, but no CRUD endpoints exist in `REST_API_SPECIFICATION`. There's no way for an admin to create, list, or revoke API keys.

Recommendation: Add API Key management endpoints: - `POST /api/v1/api-keys` — Create API key (admin only) - `GET /api/v1/api-keys` — List API keys (admin only) - `DELETE /api/v1/api-keys/{id}` — Revoke API key (admin only)

H4. WebSocket/SSE API Underspecified

`REST_API_SPECIFICATION` §8 references a “Dashboard WebSocket API” but the actual specification is missing. The `SYSTEM_ARCHITECTURE` §3.1.7 defines `NotificationService` with event types and `DashboardSession`, but there's no wire protocol spec (message formats, subscription filters, reconnection behavior, heartbeat mechanism).

Recommendation: Add a complete WebSocket API section specifying: - Connection endpoint: `wss://api.helix-ota.io/api/v1/ws/dashboard` - Authentication: JWT in first message or query param - Message format: `{type: string, payload: object, timestamp: string}` - Subscription filters: `{device_group: string, event_types: string[]}` - Heartbeat: ping/pong every 30s - Reconnection: client reconnects with `last_event_id` for missed events

H5. No Dashboard Web UI Design Document

The `VERSION_ROADMAP` §4.5 lists dashboard features (Secure Login, Upload OTA Zip, Manage Rollouts, Monitor Fleet) and `NEW_SUBMODULE_SPECIFICATIONS` §5 defines the tech stack, but there is no dedicated dashboard design document at the same level of detail as `ANDROID_CLIENT_DESIGN.md`. The dashboard is one of five major subsystems (per `SYSTEM_ARCHITECTURE` §2.2) but has the least documentation.

Recommendation: Create docs/dashboard/DASHBOARD_DESIGN.md covering: - Page-by-page design specifications - Component hierarchy and state management - API integration patterns (TanStack Query keys, WebSocket hooks) - Error handling and loading states - Responsive design breakpoints - Accessibility requirements

H6. No Go Client SDK Design Document

ANDROID_CLIENT_DESIGN.md is excellent at 800+ lines covering the Android client in depth. The Client SDK (which the Android app wraps) gets only a submodule specification in NEW_SUBMODULE_SPECIFICATIONS. This SDK is the interface between the Go server and all device-side logic — it deserves a full design document.

Recommendation: Create docs/client/CLIENT_SDK_DESIGN.md covering: - Complete interface definitions for all SDK types - gomobile AAR binding constraints and workarounds - mTLS certificate management lifecycle - Offline queue implementation details - Thread safety guarantees - Error handling taxonomy

H7. Rollout Model — SERVER_IMPLEMENTATION vs DATABASE_SCHEMA Mismatch

The Rollout struct in SERVER_IMPLEMENTATION §4.2 has: - UpdateID string — not in DATABASE_SCHEMA - Stages []RolloutStage as a Go slice — DATABASE_SCHEMA has a separate rollout_stages table - HardwareRev string — not in DATABASE_SCHEMA - MinDwellDuration time.Duration — not in DATABASE_SCHEMA - StageEnteredAt *time.Time — not in DATABASE_SCHEMA

The DATABASE_SCHEMA rollouts table has: - strategy rollout_strategy_enum — not in SERVER_IMPLEMENTATION - auto_rollback_threshold FLOAT — not in SERVER_IMPLEMENTATION - auto_rollback_enabled BOOLEAN — not in SERVER_IMPLEMENTATION

Recommendation: Reconcile the rollout model across all documents. The DATABASE_SCHEMA is the most complete — align SERVER_IMPLEMENTATION structs with it. Add the missing fields (strategy, auto_rollback_threshold, auto_rollback_enabled) to the Go model.

H8. Device Model — SYSTEM_ARCHITECTURE vs DATABASE_SCHEMA Mismatch

SYSTEM_ARCHITECTURE §3.1.2 Device struct has: - Group string with db:"device_group" tag - Status string (just a string, not typed) - LastCheckIn time.Time

DATABASE_SCHEMA devices table has: - group_id UUID (FK to device_groups) — not a string, and it's a UUID FK, not a group name - status device_status_enum — a proper enum type - last_seen_at TIMESTAMPTZ — different column name - os_type, os_version — not in

SYSTEM_ARCHITECTURE Device struct - mtlc_cert_fingerprint — not in
SYSTEM_ARCHITECTURE Device struct - ip_address_INET — not in
SYSTEM_ARCHITECTURE Device struct - target_version — in
SYSTEM_ARCHITECTURE but as *string pointer

Recommendation: Align the Go Device struct with the DATABASE_SCHEMA, including: - GroupID uuid.UUID (not Group string) - Status DeviceStatus (typed enum) - LastSeenAt time.Time (renamed from LastCheckIn) - Add missing fields: OSType, OSVersion, MTLSCertFingerprint, IPAddress

H9. No Telemetry Event Report Endpoint Specified in REST_API_SPECIFICATION

VERSION_ROADMAP defines POST /api/v1/devices/{id}/status for status reporting. SYSTEM_ARCHITECTURE §3.1.5 defines TelemetryService.IngestTelemetry(). But the REST_API_SPECIFICATION has no telemetry/event ingestion endpoint — only device status (which is different from telemetry). The telemetry event types in DATABASE_SCHEMA (check, download_start, download_progress, etc.) are richer than the device status states.

Recommendation: Add to REST_API_SPECIFICATION: - POST /api/v1/devices/{device_id}/events — Ingest telemetry events (batch supported) - GET /api/v1/telemetry/overview — Aggregated telemetry (admin/operator/viewer) - GET /api/v1/devices/{device_id}/events — Per-device event history

3. MEDIUM Issues — Should Address Before Release

M1. Artifact Download URL Signing Not Specified

SERVER_IMPLEMENTATION §3.4 mentions Download() returns a “signed, time-limited URL.” SECURITY_ARCHITECTURE doesn’t specify how URL signing works. REST_API_SPECIFICATION §4.6 shows direct download with auth headers, not signed URLs. These are fundamentally different approaches (direct auth vs pre-signed URL).

Recommendation: Choose one approach and document it. For MVP, direct download with mTLS/JWT auth is simpler. Pre-signed URLs are better for CDN/offloading. Document the decision.

M2. Certificate Authority Setup and Device Provisioning Workflow Missing

SECURITY_ARCHITECTURE §5.3 describes device certificate provisioning at a high level, but there's no operational guide for: - Setting up the Device CA - Generating device certificates - Revoking compromised certificates - Certificate renewal automation

Recommendation: Create docs/security/DEVICE_PROVISIONING_GUIDE.md with step-by-step instructions.

M3. No AOSP Build Integration Guide

ANDROID_CLIENT_DESIGN.md describes a system-privileged app installed at /system/priv-app/HelixOtaClient/, but there's no documentation on: - How to include the APK in the AOSP build - Required Android.mk/Android.bp definitions - Signing requirements for system apps - Permission allowlist configuration

Recommendation: Create docs/client/android/AOSP_INTEGRATION_GUIDE.md.

M4. No CI/CD Pipeline Documentation

TESTING_STRATEGY.md §10 references "CI/CD Integration" but the section was clipped. There's no dedicated CI/CD document specifying: - Pipeline stages (lint → test → mutation → build → deploy) - Mutation testing integration in CI - Artifact signing in CI - Hardware-in-the-loop test triggers

Recommendation: Create docs/infrastructure/CI_CD_PIPELINE.md.

M5. No Database Migration Strategy Documentation

DATABASE_SCHEMA §5 references a "Migration Strategy" section, but the actual migration files are just numbered SQL files listed in SERVER_IMPLEMENTATION §1. There's no documentation on: - Migration tool (golang-migrate is in go.mod but not explained) - Rollback procedures - Zero-downtime migration deployment - Telemetry partition creation automation

Recommendation: Expand DATABASE_SCHEMA §5 with concrete migration procedures and examples.

M6. Concurrent Rollout Targeting Overlap Not Addressed

SERVER_IMPLEMENTATION §4.5 introduces RolloutConcurrencyGuard for device assignment, but the scenario of two rollouts targeting the same device group simultaneously is not addressed in the REST_API_SPECIFICATION or DATABASE_SCHEMA. There's no validation to prevent overlapping rollouts.

Recommendation: Add a constraint: only one active rollout per device group at a time. Add a DB partial unique index: `CREATE UNIQUE INDEX uidx_rollouts_one_active_per_group ON rollouts (target_group_id) WHERE status IN ('active', 'paused');`

M7. Mandatory Update Enforcement Behavior Undefined

The update check response includes `mandatory: false` and `deadline`, but there's no specification for what the client should do when `mandatory: true` or when the deadline passes. Does the client force-install? Does it prevent the user from dismissing the notification? Does it block device usage?

Recommendation: Define mandatory update behavior: - `mandatory: true`, `deadline: null` → User cannot dismiss; update installs on next check - `mandatory: true`, `deadline: <date>` → User can postpone until deadline; after deadline, force-install - `mandatory: false` → User can dismiss indefinitely

M8. No Error Code Reference (REST_API_SPECIFICATION Appendix B)

The `REST_API_SPECIFICATION` references “Appendix B — Error Code Reference” but the document was clipped before reaching that appendix. A comprehensive error code catalog is essential for client development.

Recommendation: Complete Appendix B with all error codes organized by endpoint, including HTTP status mapping, error code strings, and human-readable messages.

4. LOW Issues — Nice to Have

L1. No Monitoring/Alerting Runbook

`DEPLOYMENT_GUIDE.md` §9 references monitoring but provides no operational runbook for common alert scenarios (high failure rate, stale devices, storage exhaustion).

L2. No Incident Response Playbook

No documented procedures for security incidents (signing key compromise, device fleet compromise, data breach).

L3. No .env.example File

`DEPLOYMENT_GUIDE.md` references environment variables extensively but there's no `.env.example` template.

L4. No Performance Budget / SLO Document

VERSION_ROADMAP specifies “Response < 200ms p95” for update checks, but there’s no comprehensive SLO document covering all endpoints, throughput targets, and error budget.

L5. No Backup/Restore Procedures

DEPLOYMENT_GUIDE.md §10 references disaster recovery but doesn’t provide concrete backup/restore procedures for PostgreSQL, MinIO, or Vault.

L6. No Internationalization (i18n) Strategy

The dashboard and client UI have no i18n plan, which is acceptable for MVP but should be noted for future versions.

L7. gomobile AAR Limitations Not Documented

The Client SDK produces an Android AAR via gomobile, but the constraints of gomobile (no chan types, no generic interfaces, limited type support) are not documented in the ANDROID_CLIENT_DESIGN.md.

5. Safety & Anti-Bluff Validation Concerns

SA1. Validation Chain Order is Safety-Critical (see C9)

The contradiction in validation chain order is not just an inconsistency — it’s a safety defect. If structure is checked before hash, a corrupted file could pass structural validation (if the corruption doesn’t break ZIP headers) but contain malicious payload. The hash check **MUST** come first to establish integrity before any semantic analysis.

Severity: CRITICAL — Must be resolved before any validation code is written.

SA2. No Anti-Rollback Protection Specified

The DATABASE_SCHEMA has no version monotonicity constraint. There’s nothing preventing a device from being “updated” to an older version. The compatibility_check validation stage is described as checking `target_version > min_source_version` but there’s no `target_version > device.current_version` constraint. A malicious or buggy server could push a downgrade.

Recommendation: Add an explicit downgrade-prevention check: `artifact.target_version > device.current_version` must be enforced both server-side (in the update check logic) and client-side (in the compatibility verifier). Add this as a hard requirement in the validation chain.

SA3. No Tampered-Artifact Server-Side Alerting

SECURITY_ARCHITECTURE specifies that devices verify artifact integrity client-side, but there's no specification for what happens when a device reports HASH_MISMATCH. Should the server automatically halt the rollout? Alert administrators? Quarantine the artifact? The anomaly detection rules in SYSTEM_ARCHITECTURE §3.1.5 mention failure-rate thresholds but don't specifically address tamper-detection events as a separate, more urgent category.

Recommendation: Define a SECURITY_TAMPER_DETECTED event type that triggers immediate rollout halt and admin notification, distinct from general failure-rate monitoring.

SA4. Anti-Bluff Validation for Documentation Itself

Per HelixConstitution §2.3, no feature is “done” until tested and documented. The documentation itself has not been validated against any acceptance criteria. There's no documented process for ensuring the documentation stays consistent as code evolves.

Recommendation: Implement a documentation validation gate in CI: - API spec is the single source of truth; generate endpoint stubs from the spec - Database schema is the single source of truth; generate Go model stubs from schema - Any manual document that contradicts these sources fails CI

SA5. Four-Layer Fix Verification Not Applied to Documentation Errors

TESTING_STRATEGY.md §1.1 requires four-layer fix verification (SOURCE → ARTIFACT → RUNTIME → USER-VISIBLE). The inconsistencies identified in this review should be treated as “bugs” and verified at all four layers: - **SOURCE:** Fix the documentation - **ARTIFACT:** Verify the generated API types match - **RUNTIME:** Verify the running server accepts/rejects the documented inputs - **USER-VISIBLE:** Verify the device client can successfully register, check, download, and update

6. Implementation Blockers Summary

A development team attempting to start implementation today would be blocked by:

#	Blocker	Affected Component	Documents to Resolve
1	Device registration endpoint & schema undefined	Server, Client	ROADMAP, REST_API, SYS_ARCH
2	Update check endpoint &	Server, Client	ROADMAP, REST_API, SYS_ARCH

#	Blocker	Affected Component	Documents to Resolve
3	method upload endpoint undefined	Server, Dashboard	ROADMAP, REST_API
4	All enum values inconsistent (status, validation, OS type)	Server, Client, DB	ALL
5	Go module path undefined	Server, all submodules	SERVER_IMPL, SUBMODULE_SPECS
6	Primary key strategy undefined (UUID vs prefixed string)	Server, DB, API	DB_SCHEMA, REST_API, SYS_ARCH
7	Rollout model doesn't match between code and DB	Server	SERVER_IMPL, DB_SCHEMA
8	Validation chain order undefined	Server	SYS_ARCH, SERVER_IMPL, SEC_ARCH

7. Recommended Resolution Priority

Phase	Issues	Timeline
Phase 1: Canonical Spec Alignment (Week 1)	C1–C11	Resolve all endpoint, schema, and enum contradictions. Produce a single authoritative API spec.
Phase 2: Missing API Surface (Week 2)	H1–H3, H9, M8	Define all missing API endpoints. Complete error code reference.
Phase 3: Model Reconciliation (Week 2)	H7, H8, C5, C6, C8 SA1–SA5, C9, M7	Align all Go structs with DB schema. Generate code from schema where possible.

Phase	Issues	Timeline
Phase 4: Safety Hardening (Week 2–3)		Fix validation chain order. Add downgrade protection. Define mandatory update behavior.
Phase 5: Documentation Completion (Week 3)	H4–H6, M1–M5, L1–L7	Write missing design docs. Complete operational guides.

8. Document-by-Document Summary

Document	Critical	High	Medium	Low	Overall Assessment
VERSION_ROADMAP	4	0	0	0	Contains endpoint definitions that contradict the REST API spec; must be updated
SYSTEM_ARCHITECTURE	3	2	0	0	Go structs don't match DB schema; validation chain order wrong
REST_API_SPECIFICATION	2	3	1	1	Most authoritative for API surface but missing key sections; appendices incomplete
DATABASE_SCHEMA	2	0	2	0	Most complete model definition but CHECK constraints conflict with API spec
SECURITY_ARCHITECTURE	0	0	2	1	High-quality threat model; missing operational guides and tamper alerting
SERVER_IMPLEMENTATION	3	1	0	0	Go module path wrong; rollout model doesn't match DB; validation chain order wrong
ANDROID_CLIENT_DESIGN	0	0	1	1	Highest quality document; missing AOSP integration and gomobile constraints
TESTING_STRATEGY	0	0	0	1	Excellent anti-bluff coverage; CI/CD section incomplete
DEPLOYMENT_GUIDE	0	0	1	2	Comprehensive Docker/K8s config; missing .env.example and backup procedures
NEW_SUBMODULE_SPECS	1	0	0	0	Module path conflicts with SERVER_IMPLEMENTATION otherwise thorough

9. Conclusion

The Helix OTA documentation suite demonstrates excellent breadth and depth of coverage — the threat model, testing strategy, and deployment guide are particularly strong. However, the documents were clearly authored by different people (or at different times) without a shared canonical specification. The result is a set of documents that individually read well but collectively contradict each other on nearly every concrete detail: API paths, HTTP methods, field names, enum values, data types, and even the order of safety-critical validation steps.

The single most important action is to designate the REST_API_SPECIFICATION and DATABASE_SCHEMA as the two sources of truth, and update all other documents to derive from them. Any document that cannot be mechanically verified against these two sources should be flagged as potentially stale.

Until the 11 CRITICAL issues are resolved, a development team will be forced to guess which document is correct — and different team members will make different guesses, leading to incompatible implementations that cannot communicate.

End of Code Review & Gap Analysis